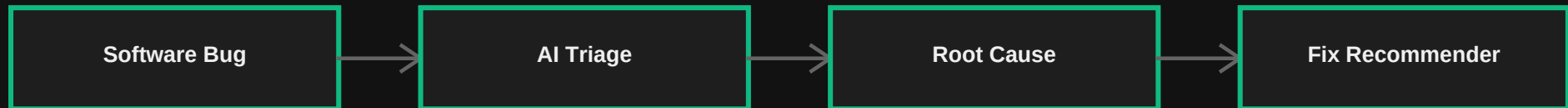


CREATION OF INTELLIGENT BUG DIAGNOSIS PLATFORM WITH FIX RECOMMENDATION ASSISTANCE GROUP BugSense AI



Slide 2: Problem Statement & Objectives

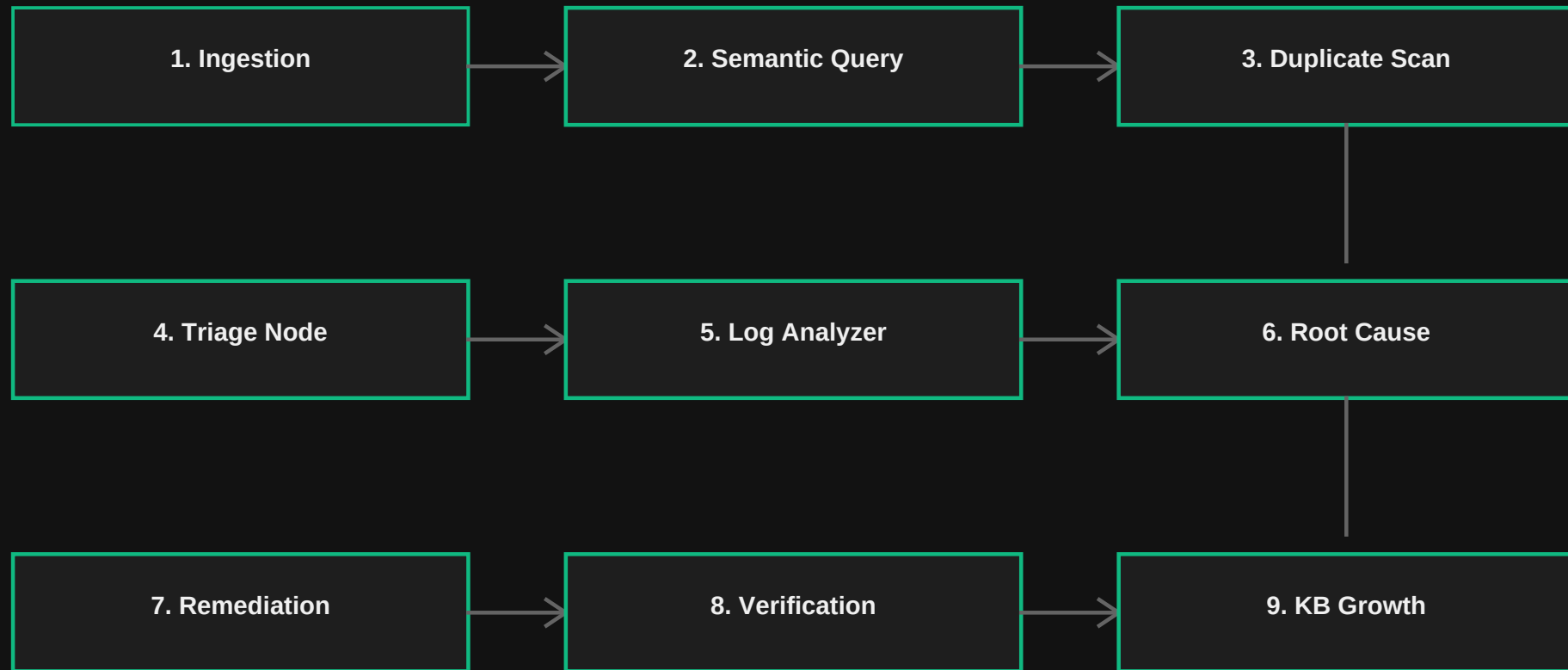
Challenges in Defect Diagnosis

- Manual stack trace parsing is slow and complex.
- Siloed resolutions lead to fixing the same bug repeatedly.
- Developers lack immediate, contextual fix recommendations.
- Verbose log files create severe cognitive overload.

Core Project Objectives

- Automate triage (severity, priority, component) and log parsing.
- Implement RAG search to leverage past solved bug patterns.
- Isolate failing file traces, methods, and stack frame classes.
- Generate fix recommendations and prevention guidelines.
- Create a closed-loop self-growing vector database mechanism.

Slide 3: Proposed Solution Workflow



* Log Ingestion: Developer uploads logs. Agent Diagnosis: Pipeline evaluates root causes. Self-Growth: Developer verifies the resolved bug, triggering the knowledge-base update process.

Slide 4: System Subsystem Topology

FRONTEND VIEW PORTAL

- React.js client interface
- Vite build configurations
- Tailwind CSS dashboard grids
- Axios proxy connections

BACKEND API GATEWAY

- Node.js + Express.js APIs
- Multer multi-part attachments
- Winston file logging wrapper
- MongoDB / local JSON fallback

AI SERVICE CORE

- FastAPI service wrappers
- LangGraph orchestrator machine
- FAISS similarity check algorithms
- Google Gemini reasoning engine

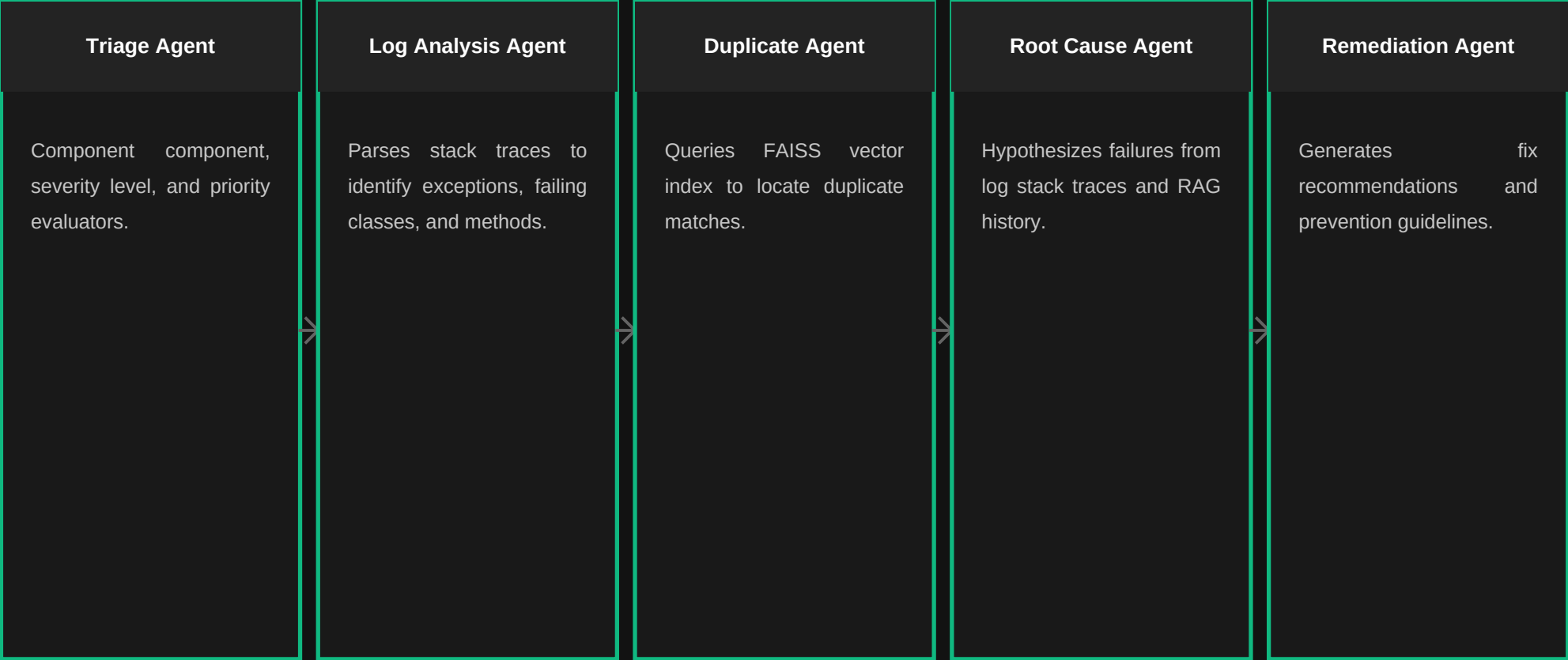
Slide 5: Technology Stack Matrix

Layer	Technology	Key Purpose
Frontend client	React, Vite, Tailwind CSS, Axios	Render diagnostic grids, charts & telemetry UI
API gateway	Node.js, Express.js, Multer, Winston	REST request proxy, log trace handling & routing
Database persistence	MongoDB (Mongoose), local JSON files	Saves bugs, verified resolutions & stats fallback
AI Microservice	Python 3.10, FastAPI	Wraps embeddings, similarity matches & agents
Vector database	FAISS, NumPy vector fallback	Maintains dense vectors for semantic similarity query
Agent runtime	LangChain, LangGraph StateGraph	Orchestrates transition state logic between agents
LLM Framework	Google Gemini API	Core inference engine for logs and recommendations

Slide 6: RAG & Semantic Retrieval Ingestion

- Vectorization Model: Computes 384-dimensional dense embeddings using sentence-transformers (all-MiniLM-L6-v2).
- FAISS Index Query: Scans indexed historical records using cosine similarity metrics to retrieve top 5 matching bug resolutions.
- RAG Context Appending: Fuses historical diagnostic details and resolutions directly into the LLM inference prompt.
- Keyword-Independent Match: Matches related defects even when code variables and keywords differ, bridging syntax discrepancies.

Slide 7: Multi-Agent Node Transition



Slide 8: Knowledge Base Growth & Verification

- Developer-Verified Gate: Resolved defects require confirmation on the dashboard. No logs are indexed immediately.

Decision Mechanism: FAISS Similarity Index Threshold (90%)

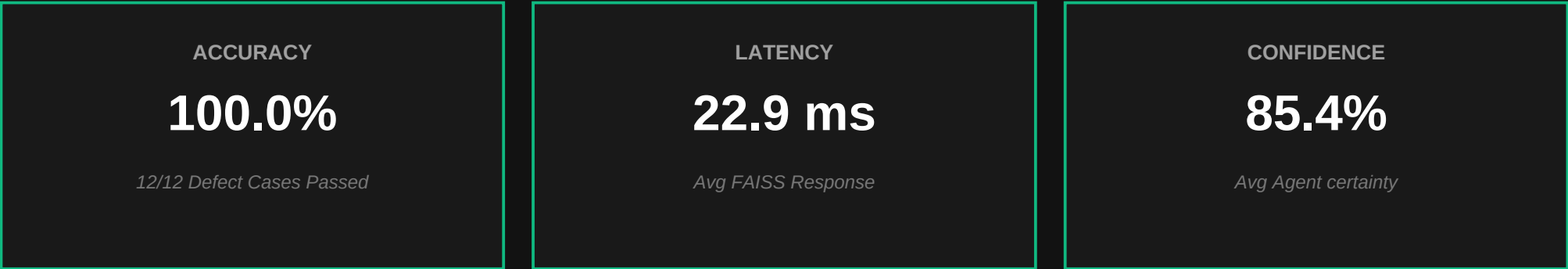
Similarity > 90% -> Update Existing

- Matches an existing database pattern.
- Enriches existing resolved metadata rather than creating redundant entries.
- Overwrites matching FAISS index labels and updates metadata fields.

Similarity <= 90% -> Create New

- Represents a new software failure mode.
- Creates a new unique vector node and inserts the playbook into MongoDB.
- Rebuilds FAISS index indices to add the new RAG mapping.

Slide 9: Project Validation Results



Metric Tested	Calculated Validation Result
Precision	100.0%
Recall	100.0%
F1 Score	100.0%
Knowledge Growth Actions	12 Created, 0 Updated (Successful growth validation)

Slide 10: Conclusion & Future Enhancements

Conclusion

- Automates defect triage and parsed log analysis.
- Bridges semantic syntax variations via RAG search query matching.
- Coordinates state graph parsing via multi-agent pipelines.
- Self-growing vector storage matches B.Tech thesis guidelines.
- Reached 100% E2E verification test case coverage successfully.

Future Enhancements

- Native VS Code / PyCharm diagnostic triggers.
- Stream real-time logs from ELK, Splunk, or Datadog agent collectors.
- Enforce RBAC user roles during Resolution Verification.
- Integrate auto-patching and code repair ML algorithms (Automated Code Repair planned as Future Work).

THANK YOU

Questions & Discussions